

DQN 기반 물류창고 배치 시뮬레이션

개인 프로젝트

이승열

창고 환경 모델링 및 단계별 보상 설계
강화학습 에이전트 구현과 결과 시각화

목차

I. 개요

II. 목표

III. 이론

IV. 실험 결과

V. 결론

VI. 고찰

VII. 향후 과제

VIII. 개인 수행 내용

DQN 기반 물류창고 배치 시뮬레이션

개인 프로젝트 / 이승열

I. 개요

물품의 종류, 무게, 수량, 단가에 따라 배치 위치를 선택하는 DQN 기반 물류창고 시뮬레이션을 구현했습니다. 1층과 2층으로 구성된 창고에서 통로와 적재 공간을 구분하고, 각 층의 배치 상태를 학습 입력으로 사용했습니다.

Online Retail 데이터의 Quantity와 UnitPrice를 전처리하고, 시뮬레이션용 Type과 Weight 속성을 추가했습니다. 물품 종류, 무게, 반품 수량, 단가 조건을 단계별로 보상 함수에 반영했으며, 에피소드별 보상 그래프와 층별 배치 화면을 통해 학습 과정을 살펴봤습니다.

II. 목표

창고의 배치 후보를 행동으로 정의하고, DQN 에이전트가 배치 규칙에 따른 누적 보상을 학습하도록 구현하는 것이 목표입니다. 여러 조건을 동시에 적용했을 때의 학습 문제를 분석하고, 조건을 순차적으로 추가하는 보상 구조를 설계했습니다. 데이터 전처리부터 환경 구성, 에이전트 학습, 결과 시각화까지 개인 개발로 수행했습니다.

III. 이론

1) Dataset 선정 및 속성 정의

시뮬레이션용 속성: 원본 데이터에 없는 Weight, Height, Width를 난수로 생성했습니다. 최종 학습에는 Weight를 사용했으며, Height와 Width는 배치 조건에서 제외했습니다.

초기 : Online Retail

Invoice	InvoiceDate	UnitPrice	CustomerID	Country
1 536385	2010-12-01 8:26	2.55	17850	United Kingdom
2 536385	2010-12-01 8:26	3.39	17850	United Kingdom
3 536385	2010-12-01 8:26	2.75	17850	United Kingdom
4 536385	2010-12-01 8:26	3.39	17850	United Kingdom
5 536385	2010-12-01 8:26	3.39	17850	United Kingdom
6 536385	2010-12-01 8:26	3.39	17850	United Kingdom
7 536385	2010-12-01 8:26	3.39	17850	United Kingdom
8 536385	2010-12-01 8:26	3.39	17850	United Kingdom
9 536385	2010-12-01 8:26	3.39	17850	United Kingdom
10 536385	2010-12-01 8:26	3.39	17850	United Kingdom
11 536387	2010-12-01 8:34	2.1	15047	United Kingdom
12 536387	2010-12-01 8:34	2.1	15047	United Kingdom
13 536387	2010-12-01 8:34	2.1	15047	United Kingdom
14 536387	2010-12-01 8:34	2.1	15047	United Kingdom
15 536387	2010-12-01 8:34	2.1	15047	United Kingdom
16 536387	2010-12-01 8:34	2.1	15047	United Kingdom
17 536387	2010-12-01 8:34	2.1	15047	United Kingdom
18 536387	2010-12-01 8:34	2.1	15047	United Kingdom
19 536387	2010-12-01 8:34	2.1	15047	United Kingdom
20 536387	2010-12-01 8:34	2.1	15047	United Kingdom
21 536387	2010-12-01 8:34	2.1	15047	United Kingdom
22 536387	2010-12-01 8:34	2.1	15047	United Kingdom
23 536387	2010-12-01 8:34	2.1	15047	United Kingdom
24 536387	2010-12-01 8:34	2.1	15047	United Kingdom
25 536387	2010-12-01 8:34	2.1	15047	United Kingdom
26 536387	2010-12-01 8:34	2.1	15047	United Kingdom
27 536387	2010-12-01 8:34	2.1	15047	United Kingdom
28 536387	2010-12-01 8:34	2.1	15047	United Kingdom
29 536387	2010-12-01 8:34	2.1	15047	United Kingdom
30 536387	2010-12-01 8:34	2.1	15047	United Kingdom
31 536387	2010-12-01 8:34	2.1	15047	United Kingdom
32 536387	2010-12-01 8:34	2.1	15047	United Kingdom
33 536387	2010-12-01 8:34	2.1	15047	United Kingdom
34 536387	2010-12-01 8:34	2.1	15047	United Kingdom
35 536387	2010-12-01 8:34	2.1	15047	United Kingdom
36 536387	2010-12-01 8:34	2.1	15047	United Kingdom
37 536387	2010-12-01 8:34	2.1	15047	United Kingdom

기존 초기 Dataset에서
Quantity & UnitPrice 정보 추출



+) 자체 정보
Type & Weight 정보 추가

TYPE	Quantity	UnitPrice	weight
PROUDCT	6	2.55	0.85
RETURN	6	3.39	3.75
PROUDCT	8	2.75	3.75
PROUDCT	6	3.39	0.85
RETURN	6	3.39	1.65
PROUDCT	2	7.65	2.55
PROUDCT	6	4.25	2.55
PROUDCT	6	1.85	0.42
RETURN	6	1.85	0.55
PROUDCT	32	1.69	0.55
PROUDCT	6	2.1	0.55
RETURN	6	2.1	2.95
PROUDCT	8	3.75	1.25
PROUDCT	6	1.65	0.38
PROUDCT	6	4.25	1.95
RETURN	3	4.95	1.95
PROUDCT	2	9.95	1.45
PROUDCT	3	5.95	4.25
RETURN	3	5.95	4.95

데이터 전처리: 판매량과 단가를 추출합니다. 판매량과 단가의 열을 숫자형으로 변환, 불가능한 값은 제거합니다. 판매량이 0이 아니고 단가가 양수인 데이터만 유지합니다. 판매량이 0이 아닌 값을 유지하는 이유는 반품 상품을 포함하기 때문입니다.

```
import os
import pandas as pd

def convert_excel_to_csv(excel_path, csv_path):
    try:
        data = pd.read_excel(excel_path, engine="openpyxl")
        data.to_csv(csv_path, index=False, encoding="utf-8-sig")
        print(f"Excel 파일이 CSV로 변환되었습니다: {csv_path}")
    except Exception as e:
        print(f"Excel 변환 중 오류 발생: {e}")

def process_online_retail_csv(csv_path, output_path):
    try:
        retail_data = pd.read_csv(csv_path)
        retail_data["Quantity"] = pd.to_numeric(retail_data["Quantity"], errors="coerce")
        retail_data["UnitPrice"] = pd.to_numeric(retail_data["UnitPrice"], errors="coerce")
        retail_data["StockCode"] = retail_data["StockCode"].astype(str).str.strip()
        retail_data.dropna(subset=["Quantity", "UnitPrice"], inplace=True)
        retail_data = retail_data[(retail_data["Quantity"] != 0) & (retail_data["UnitPrice"] > 0)]
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
        retail_data.to_csv(output_path, index=False, encoding="utf-8-sig")
        print(f"데이터 분석 결과가 저장되었습니다: {output_path}")
        return retail_data
    except Exception as e:
        print(f"데이터 처리 중 오류 발생: {e}")
        return None

excel_file = "C:/project/final/datasets/online+retail/Online Retail.xlsx"
csv_file = "C:/project/final/datasets/online+retail/Online Retail.csv"
output_file = "C:/project/final/datasets/preprocessing/Online_Retail_Cleaned.csv"

try:
    convert_excel_to_csv(excel_file, csv_file)
    processed_data = process_online_retail_csv(csv_file, output_file)
    if processed_data is not None:
        print("\nOnline Retail Cleaned Data (샘플):")
        print(processed_data.head())
    else:
        print("데이터 분석에 실패했습니다.")
except Exception as e:
    print(f"전체 과정 중 오류 발생: {e}")
```

2) 창고 환경 시각화 및 모델링 (WarehouseEnv)

창고 환경은 2층 구조로 설계되었습니다.

이동 경로 및 적재함: 숫자 '0'은 작업자나 로봇이 이동할 수 있는 통로를 의미하며, '1'은 물품을

적재할 수 있는 슬롯을 나타냅니다.

```
class WarehouseEnv:
    def __init__(self, combined_states):
        self.states = combined_states
        self.layout = np.array([
            [1, 1, 1, 1, 1, 1, 1], # 0번줄
            [0, 0, 0, 0, 0, 0, 0],
            [1, 1, 1, 1, 1, 1, 1], # 2번줄
            [0, 0, 0, 0, 0, 0, 0],
            [1, 1, 1, 1, 1, 1, 1], # 4번줄
            [0, 0, 0, 0, 0, 0, 0],
            [1, 1, 0, 0, 0, 1, 1], # 6번줄
            [1, 1, 0, 0, 0, 1, 1], # 7번줄
            [1, 1, 0, 0, 0, 1, 1], # 8번줄
            [0, 0, 0, 0, 0, 0, 0]
        ], dtype=int)
        self.max_weight_1f = 100
        self.max_weight_2f = 50
        self.state_1f = np.zeros_like(self.layout, dtype=np.float32)
        self.state_2f = np.zeros_like(self.layout, dtype=np.float32)
        self.items_1f = np.empty(self.layout.shape, dtype=object)
        self.items_2f = np.empty(self.layout.shape, dtype=object)
        self.total_reward = 0
        self.valid_positions = np.argwhere(self.layout == 1).tolist()
        self.current_item_type = "Product" # 초기 타입
```

창고 좌표 배열: 각 층의 배치 상태와 물품 속성을 관리했습니다. 단가 조건은 0번 행에서 4 이상, 2번 행에서 2 이상, 4번 행에서 2 미만일 때 보상을 부여하도록 정의했습니다.

반품 구역: 6, 7, 8번 행의 지정된 슬롯을 Return Zone으로 설정하고, 수량 조건과 행 위치에 따라 보상을 부여했습니다.

무게 조건: 1층에는 50~200kg, 2층에는 0~50kg 조건을 적용했습니다.
빈 슬롯에 배치할 때 기본 보상 0.5를 부여했습니다.

3) Reward 함수 설계

초기에는 모든 조건을 동시에 적용하였으나 학습이 제대로 이루어지지 않았습니다. 이를 해결하기 위해 **단계별 Reward** 구조로 개선하였습니다.

1단계: Type 기반 물건 배치

반품 물건은 Return zone에, 일반 물건은 Return zone이 아닌 곳에 배치하여 Reward와 페널티를 주었습니다.

```
# 1. 빈 공간에 물건이 들어가면 보상
if self.state_1f[x, y] == 0 or self.state_2f[x, y] == 0:
    reward += 0.5 # 배치된 빈 공간에 대해 보상

# 1단계: 리턴 존과 프로덕트 위치 조건 추가
if step_level >= 1:
    if item_type == "Return Zone":
        if (x, y) in [(6, 0), (6, 1), (7, 0), (7, 1), (8, 0), (8, 1)]:
            reward += 5 # 리턴 존이 올바른 위치에 배치
        else:
            reward -= 10 # 잘못된 위치에 패널티 강화
    elif item_type == "Product":
        if (x, y) in [(6, 0), (6, 1), (7, 0), (7, 1), (8, 0), (8, 1)]:
            reward -= 5 # 프로덕트가 잘못된 위치에 배치
        else:
            reward += 10 # 프로덕트가 올바른 위치에 배치
```

2단계: 무게 기반 배치

각 층의 무게 조건을 만족하면 보상 1을, 만족하지 않으면 보상 -1을 부여했습니다. 배치 후보와 물품 무게의 관계를 학습하도록 구성했습니다.

3단계: 반품량 기반 배치

반품량 기준 30, 15, 1이상 시 각각 1, 0.7 0.5로 차등 보상 지급됩니다.

```

# 2단계: 무게 조건
if step_level >= 2:
    if x < 5: # 1층
        reward += 1.0 if 50 <= weight <= 200 else -1.0
    else: # 2층
        reward += 1.0 if 0 <= weight <= 50 else -1.0

# 3단계: 판매량 조건
if step_level >= 3:
    if (x, y) in [(8, 0), (8, 1)]:
        reward += 1.0 if quantity >= 30 else 0
    elif (x, y) in [(7, 0), (7, 1)]:
        reward += 0.7 if quantity >= 15 else 0
    elif (x, y) in [(6, 0), (6, 1)]:
        reward += 0.5 if quantity >= 1 else 0

```

4단계: 물품 가격 기반 배치

단가는 0번 행에서 4 이상, 2번 행에서 2 이상, 4번 행에서 2 미만일 때 각각 보상 1, 0.7, 0.5를 부여했습니다. 가격 조건을 만족하는 위치를 학습하도록 구성했습니다.

```

# 4단계: 가격 조건
if step_level >= 4:
    if x == 0:
        reward += 1.0 if unit_price >= 4 else 0
    elif x == 2:
        reward += 0.7 if unit_price >= 2 else 0
    elif x == 4:
        reward += 0.5 if unit_price < 2 else 0

# 보상 범위 제한 (최대 보상 값 설정)
max_reward = 15 # 최대 보상 값
min_reward = -15 # 최소 보상 값
reward = np.clip(reward, min_reward, max_reward)

# 상태 업데이트
# 물건 배치
if 50 <= weight <= 200: # 1층 배치
    if self.state_1f[x, y] + weight <= self.max_weight_1f:
        self.state_1f[x, y] += weight
        self.items_1f[x, y] = item
elif 0 <= weight <= 50: # 2층 배치
    if self.state_2f[x, y] + weight <= self.max_weight_2f:
        self.state_2f[x, y] += weight
        self.items_2f[x, y] = item

```

4) DQN agent 설계

네트워크 구조: 2개의 은닉층과 1개의 출력층으로 구성된 신경망을 사용하여 Q-값을 예측합니다.

```
def build_model(self):
    model = Sequential([
        Dense(64, input_dim=self.state_size, activation='relu'),
        Dense(64, activation='relu'),
        Dense(self.action_size, activation='linear')
    ])
    model.compile(loss='mse', optimizer=Adam(learning_rate=0.001))
    return model

def act(self, state):
    if np.random.rand() <= self.epsilon:
        return random.randrange(self.action_size)
    act_values = self.model.predict(state, verbose=0)
    return np.argmax(act_values[0])
```

```
# DQN 에이전트 정의
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=12000)
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.999
        self.model = self.build_model()
        self.target_model = self.build_model() # 타겟 네트워크 생성
        self.update_target_model()
```

주 네트워크는 참고 상태를 입력으로 받아 각 배치 후보의 Q값을 출력합니다. 두 개의 은닉층은 각각 64개의 노드와 ReLU를 사용하고, 출력층은 선형 활성화 함수를 사용했습니다. 학습에는 MSE 손실 함수와 Adam 옵티마이저를 적용했습니다.

Q값은 특정 상태에서 행동을 선택했을 때 기대되는 할인 누적 보상입니다. 에이전트는 epsilon-greedy 방식으로 무작위 탐험과 Q값 기반 행동 선택을 수행했습니다. 주 네트워크의 가중치를 복사한 타겟 네트워크를 TD 목표값 계산에 사용했습니다.

경험 재생: 상태 전이를 저장하는 메모리 용량을 3,000개에서 12,000개로 늘렸습니다. 무작위로 추출한 경험을 학습에 사용하도록 구성했습니다.

```
def update_target_model(self):
    self.target_model.set_weights(self.model.get_weights())

def remember(self, state, action, reward, next_state, done):
    self.memory.append((state, action, reward, next_state, done))

def replay(self, batch_size):
    minibatch = random.sample(self.memory, batch_size)
    for state, action, reward, next_state, done in minibatch:
        self._update_model(state, action, reward, next_state, done)

    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

    # 타깃 네트워크 주기적 업데이트
    if random.random() < 0.1: # 10% 확률로 업데이트
        self.update_target_model()
```

update_target_model: 주 네트워크의 가중치를 타깃 네트워크로 복사합니다.

remember: 현재 상태, 행동, 보상, 다음 상태, 종료 여부를 저장합니다.

replay: 저장된 경험에서 batch_size개를 무작위로 추출해 학습합니다. 탐험 확률 epsilon을 감소시키며 행동 선택 비율을 조정합니다.

```
def _update_model(self, state, action, reward, next_state, done):
    target = reward
    if not done:
        target += self.gamma * np.amax(self.target_model.predict(next_state, verbose=0)[0]) # 타깃 네트워크 사용
    target_f = self.model.predict(state, verbose=0)
    target_f[0][action] = target
    self.model.fit(state, target_f, epochs=1, verbose=0)

    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay
```

TD 목표값은 즉시 보상과 할인한 다음 상태의 최대 Q값으로 계산합니다. 종료 상태에서는 즉시 보상을 목표값으로 사용합니다.

```

# 실시간 보상 그래프
def plot_rewards(rewards):
    plt.figure(figsize=(10, 5))
    plt.plot(rewards, label="Episode Reward")
    plt.xlabel("Episodes")
    plt.ylabel("Total Reward")
    plt.title("Reward Progression Over Episodes")
    plt.legend()
    plt.grid()
    plt.show()

# GIF 저장 함수
def save_gif(frames, path, episode):
    os.makedirs(path, exist_ok=True)
    gif_path = os.path.join(path, f"episode_{episode}.gif")
    imageio.mimsave(gif_path, frames, fps=2)
    print(f"GIF 저장 완료: {gif_path}")

# 최종 보상 그래프 저장 함수
def plot_rewards(rewards, save_path):
    plt.figure(figsize=(10, 5))
    plt.plot(rewards, label="Episode Reward")
    plt.xlabel("Episodes")
    plt.ylabel("Total Reward")
    plt.title("Reward Progression Over Episodes")
    plt.legend()
    plt.grid()
    plt.savefig(os.path.join(save_path, "final_rewards_graph.png")) # 그래프 저장
    plt.show()

```

실시간 보상 그래프와 학습과정과 추세를 보기 위한 gif 저장과, 최종 보상 그래프 저장을 합니다.

```

# 실행 코드
try:
    with open(combined_states_path, "r", encoding="utf-8") as f:
        combined_states = json.load(f)

    env = WarehouseEnv(combined_states)
    state_size = len(env.state_1f.flatten()) + len(env.state_2f.flatten())
    action_size = len(env.valid_positions)
    agent = DQNAgent(state_size, action_size)

    num_episodes = 300 # 총 에피소드 수
    max_steps = 450 # 한 에피소드당 최대 스텝 수
    episodes_per_level = num_episodes // 4 # 레벨당 에피소드 수 (총 4단계)
    max_step_level = 4 # 최대 step_level

    all_rewards = []
    episode_rewards = []

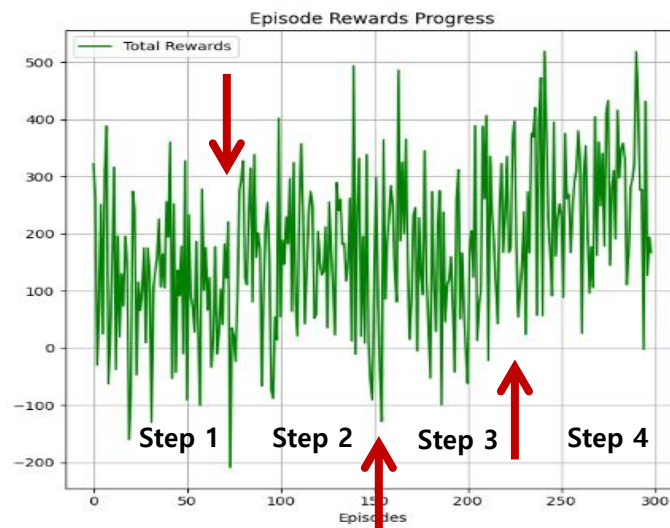
    # GIF 저장 경로 및 그래프 저장 경로
    gif_output_path = "C:/project/final/datasets/sample_episodes"
    graph_output_path = "C:/project/final/datasets/graph"

    # 실시간 시각화 플롯 생성
    fig, axes = plt.subplots(1, 3, figsize=(18, 8)) # 1F, 2F, 보상 그래프
    plt.ion()

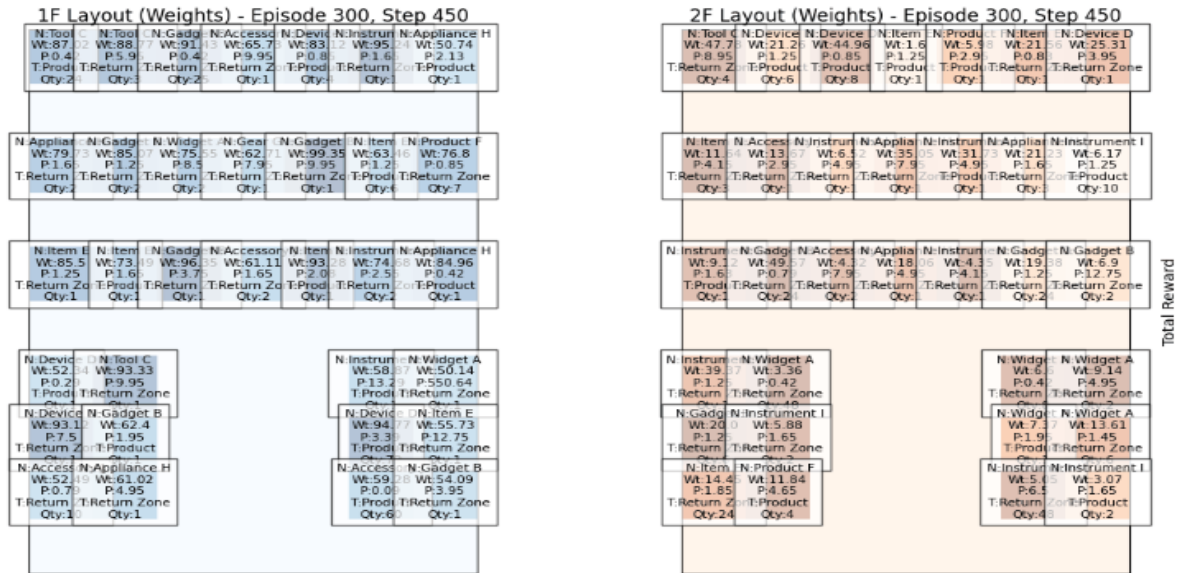
```

강화 학습 환경을 초기화하고, 학습을 수행하기 위한 기본적인 파라미터와 경로를 설정합니다. 에이전트는 정의된 환경에서 학습하며, 결과는 GIF와 그래프로 저장되어 학습의 효과를 분석할 수 있습니다.

IV. 실험 결과



에피소드별 총 보상을 기록한 그래프입니다. 300개 에피소드를 75개씩 나누고, 각 단계에서 물품 종류, 무게, 반품 수량, 단가 조건을 순차적으로 추가했습니다. 후반 구간에서 보상이 높아지는 경향을 관찰했습니다. 단계마다 보상 기준이 달라지므로 구간별 총 보상은 동일한 평가 기준의 성능 비교와 구분해 해석했습니다.



1층과 2층의 배치 상태를 시각화한 화면입니다. 색이 채워진 칸은 물품이 배치된 슬롯이며, 각 칸에 물품의 단가, 수량, 종류와 무게를 표시했습니다. 이를 통해 에이전트가 선택한 위치와 물품 속성을 함께 확인할 수 있도록 구성했습니다.

V. 결론

2층 창고 환경과 DQN 에이전트를 연결하고, 물품 특성에 따른 배치 규칙을 보상 함수로 구현했습니다. 처음에는 여러 조건을 동시에 적용했으나 보상의 변동이 컸고, 조건을 단계적으로 추가하는 구조로 학습 방식을 조정했습니다. 경험 재생 메모리를 12,000개로 설정하고 타깃 네트워크를 사용했습니다.

학습 과정은 보상 그래프와 배치 화면으로 기록했습니다. 이 프로젝트에서는 창고 배치 문제를 상태와 행동으로 모델링하고, 보상 설계에 따라 학습 과정을 관찰하는 구현 경험을 얻었습니다. 실제 물류 운영에서의 효율 향상은 고정된 평가 조건과 비교 정책을 적용한 후속 실험 과제로 남겼습니다.

VI. 고찰

여러 배치 조건을 한 번에 반영했을 때 음수 보상이 자주 발생하고, 에피소드별 보상의 변동도 컸습니다. 물품 종류 조건부터 시작해 무게, 반품 수량, 단가 조건을 차례로 추가하도록 보상 함수를 구성했습니다. 각 단계에서 적용하는 규칙을 분리해 학습 과정을 살펴봤습니다.

경험 재생 메모리는 3,000개에서 12,000개로 확장했습니다. 저장할 수 있는 상태 전이의 수를 늘리고, 무작위 표본을 주 네트워크 학습에 사용했습니다. 메모리 용량의 영향은 동일한 데이터와 보상 조건에서 비교할 후속 실험 항목으로 정리했습니다.

학습 그래프를 해석할 때에는 보상 함수 자체의 변화도 함께 고려해야 했습니다. 총 보상뿐 아니라 규칙 충족률과 잘못된 배치 비율을 함께 평가하는 방향으로 실험 설계를 확장할 수 있습니다.

VII. 향후 과제

고정된 평가 조건: 학습이 끝난 정책에 같은 물품 목록과 보상 기준을 적용하고, 무작위 배치 또는 규칙 기반 배치와 비교하는 것이 다음 과제입니다. 규칙 충족률, 잘못된 배치 비율, 미배치 물품 수를 함께 측정할 수 있습니다.

주문 날짜 활용: 기존 데이터의 날짜 정보를 배치와 출고 순서에 연결하면, 시간에 따른 물품 흐름과 선입선출 조건을 고려한 환경으로 확장할 수 있습니다.

3차원 적재: 현재는 슬롯과 무게 중심으로 환경을 구성했습니다. 길이, 폭, 높이와 적재 가능 부피를 함께 반영하는 3차원 배치 문제는 추가 구현 과제입니다.

VIII. 개인 수행 내용

이승열 / 개인 개발

데이터 전처리 및 시뮬레이션 속성 구성, WarehouseEnv 환경 모델링, 단계별 보상 함수와 DQN 에이전트 구현, 학습 그래프 및 배치 시각화, 결과 분석과 발표 자료 작성을 수행했습니다.